29.3.3 Adding CORS to specific endpoints with EnableCors metadata

Browsers block cross-origin requests by default for good reason: they have the potential to be abused by malicious or compromised sites. Enabling CORS for your entire app may not be worth the risk if you know that only a subset of actions will ever need to be accessed cross-origin.

If that's the case, it's best to enable a CORS policy only for those specific endpoints. ASP.NET Core provides the `RequireCors()` method, which you can apply to your minimal API endpoints or route groups, and the `[EnableCors]` attribute, which lets you select a policy to apply to a given controller or action method.

NOTE Both these methods add CORS metadata to the endpoint, which is used by the `CorsMiddleware` to determine the policy to apply. This is why the `CorsMiddleware` should be placed after the `RoutingMiddleware`, so that the `CorsMiddleware` knows which endpoint was selected and so which CORS policy to apply.

With the `RequireCors()` method and `[EnableCors]` attribute, you can apply different CORS policies to different endpoints. For example, you could allow `GET` requests access to your entire API from the http://shopping.com domain but allow other HTTP verbs only for a specific endpoint while allowing anyone to access your product list endpoints.

You define CORS policies in the call to `AddCors()` by calling `AddPolicy()` and giving the policy a name, as you saw in listing 29.2. If you're using endpoint-specific policies, instead of calling `UseCors("AllowShoppingApp")` as you saw in listing 29.3, you